

DOKUMENTASI IMPLEMENTASI

APLIKASI MOVIE STREAMING

React JS • Golang Fiber/gRPC • PostgreSQL • Redis • Docker

Versi Dokumen: 1.0 Tanggal: 30 September 2026

Dokumen ini menjadi blueprint teknis untuk pengembangan aplikasi movie dengan tiga role utama: SuperAdmin, Admin, dan Viewer. Arsitektur dirancang modular agar setiap domain dapat dikembangkan, diuji, di-scale, dan dideploy secara independen.

Komponen	Teknologi
Frontend	React JS
API Gateway / HTTP	Golang Fiber
Internal Service Communication	gRPC
Cache / Session / Queue ringan	Redis
Database	PostgreSQL
Containerization	Docker + Docker Compose
Media Processing	FFmpeg worker/service
Authentication	JWT + CAPTCHA + Refresh Token

1. Tujuan dan Ruang Lingkup

Sistem menyediakan katalog dan pemutaran movie, manajemen kategori, upload dan pemrosesan video, subtitle/terjemahan, iklan sebelum pemutaran, serta kanal dukungan berbasis QRIS. Hak akses dikontrol dengan RBAC.

Target utama:

- Viewer: menjelajah katalog, melihat detail movie, memilih kualitas, subtitle, dan memutar film.
- Admin: mengelola movie, kategori, subtitle, metadata, dan proses upload/transcoding.
- SuperAdmin: seluruh hak Admin ditambah manajemen akun, role, konfigurasi iklan, QRIS, audit, dan penghapusan data.
- Sistem media mendukung beberapa resolusi, termasuk 4K bila source dan resource transcoding memungkinkan.
- Komunikasi antar service menggunakan gRPC; akses client menggunakan REST/HTTP melalui Fiber API Gateway.

Catatan 4K: fitur "diperkuat ke 4K" sebaiknya dipahami sebagai dukungan playback/transcoding sampai 2160p. Upscaling source 1080p menjadi 4K tidak otomatis meningkatkan detail asli dan memerlukan pipeline khusus jika memang dibutuhkan.

2. Arsitektur Sistem

Arsitektur direkomendasikan menggunakan API Gateway di depan domain services. React tidak mengakses database secara langsung.

Diagram logis:

```
React JS
↓ HTTPS
Golang Fiber API Gateway
■■■■ Auth Service ■■■■■■ PostgreSQL / Redis
■■■■ User Service ■■■■■■ PostgreSQL
■■■■ Movie Service ■■■■■■ PostgreSQL / Object Storage
■■■■ Category Service ■■■■■■ PostgreSQL
■■■■ Media Service ■■■■■■ FFmpeg / Worker / Storage
■■■■ Subtitle Service ■■■■■■ PostgreSQL / Storage
■■■■ Ad Service ■■■■■■ PostgreSQL / Redis
■■■■ Support/QRIS Service ■■■■ PostgreSQL / Storage
■■■■ Audit Service ■■■■■■ PostgreSQL / Redis
```

Internal communication: gRPC

Rekomendasi production: file video, thumbnail, subtitle, dan QRIS disimpan di object storage (S3-compatible/MinIO atau cloud storage), sedangkan PostgreSQL menyimpan metadata dan Redis menangani cache/short-lived state.

Komponen dan tanggung jawab:

Service	Tanggung Jawab	Teknologi
API Gateway	REST API, JWT middleware, rate limit, routing	Fiber
Auth	Login, CAPTCHA verification, token, refresh/revoke	Go + gRPC
User	CRUD akun, role, status akun	Go + gRPC
Movie	Metadata, katalog, publish/unpublish	Go + gRPC
Category	CRUD kategori/genre	Go + gRPC
Media	Upload, transcoding, HLS/MP4, thumbnail	Go + FFmpeg
Subtitle	Upload dan mapping subtitle per movie	Go + gRPC
Ad	Campaign iklan dan keputusan penayangan	Go + Redis
QRIS/Support	Upload QRIS dan halaman dukungan	Go + Storage

Service	Tanggung Jawab	Teknologi
Audit	Log aksi administratif dan keamanan	Go + PostgreSQL

3. Role dan Authorization

Fitur	SuperAdmin	Admin	Viewer
Login / Logout	✓	✓	✓
Lihat katalog / detail	✓	✓	✓
Putar movie	✓	✓	✓
Pilih kualitas / subtitle	✓	✓	✓
CRUD kategori	✓	✓	—
Upload movie	✓	✓	—
Transcoding / publish	✓	✓	—
Edit / hapus movie	✓	✓*	—
Kelola subtitle	✓	✓	—
Kelola iklan	✓	Opsional	—
Upload / ubah QRIS	✓	Opsional	—
CRUD akun	✓	—	—
Ubah role / suspend akun	✓	—	—
Audit log	✓	Terbatas	—
Konfigurasi sistem	✓	—	—

*Hak Admin untuk edit/hapus dapat dipersempit melalui permission matrix jika dibutuhkan. SuperAdmin tetap menjadi role dengan akses penuh.

4. Service Authentication

Alur login:

- 1 React meminta CAPTCHA challenge/token dari provider CAPTCHA.
- 2 User mengirim username/email, password, dan CAPTCHA token ke API Gateway.
- 3 Gateway meneruskan request ke Auth Service melalui gRPC.
- 4 Auth Service memvalidasi CAPTCHA, kredensial, status akun, dan role.
- 5 Password diverifikasi menggunakan Argon2id atau bcrypt; password plaintext tidak pernah disimpan.
- 6 Server menerbitkan access token JWT berumur pendek dan refresh token yang dapat direvoke.
- 7 Refresh token disimpan secara aman (idealnya httpOnly secure cookie atau rotation strategy); Redis dapat digunakan untuk blacklist/revocation.

Contoh claims JWT:

```
{ "sub": "user-uuid", "role": "ADMIN", "permissions": ["movie:create", "movie:update"], "iat": 1727660000, "exp": 1727663600, "jti": "token-uuid" }
```

Endpoint publik:

Method	Endpoint	Keterangan
POST	/api/v1/auth/login	Login + CAPTCHA
POST	/api/v1/auth/refresh	Refresh access token
POST	/api/v1/auth/logout	Revoke refresh token
GET	/api/v1/auth/me	Profil dan role

Method	Endpoint	Keterangan
POST	/api/v1/auth/captcha/verify	Opsional jika verifikasi dipisah

5. Movie Service dan Category Service

Movie Service mengelola metadata, status publikasi, relasi kategori, poster, trailer, media asset, dan statistik dasar. Category Service mengelola genre/category agar tidak hard-coded di frontend.

Contoh data movie:

id, title, slug, synopsis, release_year, duration_seconds, rating, poster_url, backdrop_url, status, created_by, published_at, created_at, updated_at

Status movie yang disarankan: DRAFT → PROCESSING → READY → PUBLISHED, dengan FAILED sebagai status error transcoding/upload.

Method	Endpoint	Role
GET	/api/v1/movies	Public/Viewer
GET	/api/v1/movies/:slug	Public/Viewer
POST	/api/v1/movies	Admin/SuperAdmin
PATCH	/api/v1/movies/:id	Admin/SuperAdmin
DELETE	/api/v1/movies/:id	Admin/SuperAdmin*
POST	/api/v1/movies/:id/publish	Admin/SuperAdmin
GET	/api/v1/categories	Public/Viewer
POST	/api/v1/categories	Admin/SuperAdmin
PATCH	/api/v1/categories/:id	Admin/SuperAdmin
DELETE	/api/v1/categories/:id	Admin/SuperAdmin

*Untuk data yang sudah pernah diputar, soft delete lebih aman daripada hard delete agar audit dan histori tetap konsisten.

6. Media Upload, Compression dan Transcoding

Upload movie tidak sebaiknya diproses langsung di HTTP request. Gunakan asynchronous job agar API tetap responsif.

Pipeline:

- 1 Admin/SuperAdmin meminta upload session dan mendapatkan upload URL/chunk session.
- 2 Frontend mengunggah file langsung ke object storage atau melalui upload service.
- 3 Media Service mencatat asset dengan status UPLOADED dan membuat job transcoding.
- 4 Worker mengambil job dari Redis queue/stream atau mekanisme job yang dipilih.
- 5 FFmpeg melakukan probing, validasi codec, ekstraksi metadata, thumbnail, dan transcoding.
- 6 Output dibuat dalam beberapa profile, misalnya 480p, 720p, 1080p, dan 2160p jika source/resource mendukung.
- 7 Untuk streaming, hasil utama direkomendasikan HLS dengan playlist master dan segment per resolusi.
- 8 Movie berubah menjadi READY setelah semua asset wajib tervalidasi; Admin kemudian dapat publish.

Contoh profile:

Profile	Resolusi	Kegunaan
SD	854×480	Bandwidth rendah
HD	1280×720	Perangkat mobile / jaringan sedang
FHD	1920×1080	Default kualitas tinggi

Profile	Resolusi	Kegunaan
UHD	3840×2160	4K; membutuhkan source dan resource memadai

Catatan: compression harus mempertimbangkan codec, bitrate, frame rate, audio, dan target device. Jangan menghapus source original sebelum seluruh output tervalidasi dan kebijakan retensi diterapkan.

7. Subtitle / Terjemahan Movie

Subtitle Service menangani file SRT/VTT dan metadata bahasa. Untuk web playback, VTT umumnya praktis untuk HTML5/HLS.

Field	Contoh
movie_id	uuid
language_code	id / en / ja
label	Bahasa Indonesia
file_url	storage URL
is_default	true/false
status	ACTIVE/INACTIVE

Endpoint: POST /api/v1/movies/:id/subtitles, GET /api/v1/movies/:id/subtitles, PATCH /api/v1/subtitles/:id, DELETE /api/v1/subtitles/:id.

Jika yang dimaksud "terjemahan" adalah terjemahan otomatis, tambahkan Translation Worker terpisah agar proses machine translation tidak membebani Movie Service. Hasil terjemahan harus dapat direview sebelum dipublish.

8. Advertisement Service

Kebutuhan: iklan muncul setiap kali user memulai pemutaran film. Implementasi sebaiknya menggunakan Ad Decision endpoint sehingga frontend tidak menentukan sendiri iklan mana yang tampil.

Alur:

- 1 Viewer membuka detail movie.
- 2 Saat tombol Play ditekan, frontend meminta POST /api/v1/ads/decision.
- 3 Ad Service memeriksa campaign aktif, targeting, frequency rule, dan availability.
- 4 Service mengembalikan creative/URL dan tipe iklan, misalnya pre-roll.
- 5 Frontend memutar iklan dan mengirim event impression/completed/click.
- 6 Setelah kondisi selesai terpenuhi, playback movie dimulai.

Redis dapat menyimpan counter/frequency state, misalnya key ad:frequency:{user/device}:{campaign}, dengan TTL sesuai aturan campaign.

Untuk menghindari iklan diputar berulang akibat refresh atau reconnect, simpan playback/ad session dengan TTL dan gunakan idempotency key.

9. Support / QRIS Service

SuperAdmin dapat mengunggah QRIS yang ditampilkan pada halaman dukungan. Admin dapat diberi permission khusus bila organisasi menginginkannya.

Data:

Field	Keterangan
id	UUID
title	Judul dukungan
description	Pesan dukungan

Field	Keterangan
qris_url	Lokasi file QRIS
is_active	Status aktif
created_by	User uploader
created_at / updated_at	Audit timestamp

Validasi upload: MIME type, ukuran file maksimum, ekstensi yang diizinkan, checksum, dan pemeriksaan file. Jangan menyimpan file upload dengan nama asli sebagai path publik.

10. Dashboard

Dashboard menampilkan data berbeda sesuai role.

Role	Widget yang disarankan
SuperAdmin	Total user, movie, category, storage, job gagal, playback, iklan, support, audit/security events
Admin	Movie draft/processing/ready, upload progress, movie terbaru, category, subtitle, job gagal
Viewer	Continue watching (jika diaktifkan), rekomendasi, genre, recently added

Data dashboard sebaiknya berasal dari endpoint agregasi atau read model, bukan membuat frontend memanggil puluhan endpoint secara paralel.

11. Desain Database PostgreSQL

Tabel inti yang disarankan:

Tabel	Kolom inti / fungsi
users	id, email, password_hash, name, role, status, last_login_at
roles / permissions	Jika RBAC ingin granular
categories	id, name, slug, description, status
movies	id, title, slug, synopsis, year, duration, status, poster, backdrop
movie_categories	movie_id, category_id
media_assets	movie_id, type, resolution, codec, bitrate, url, status
media_jobs	asset_id, job_type, status, progress, error_message
subtitles	movie_id, language, label, url, default, status
ad_campaigns	name, status, start_at, end_at, priority, frequency_rule
ad_creatives	campaign_id, type, media_url, target_url
ad_events	session_id, campaign_id, event_type, occurred_at
support_qris	title, description, qris_url, is_active
refresh_tokens	user_id, token_hash, expires_at, revoked_at
audit_logs	actor_id, action, resource, resource_id, metadata, created_at
playback_sessions	user_id/device, movie_id, started_at, ended_at

Gunakan UUID sebagai primary key untuk entity publik. Tambahkan index pada slug, status, category relation, published_at, movie_id, dan field yang sering digunakan pada pencarian/filter.

12. Kontrak gRPC Antar-Service

Setiap service memiliki .proto sendiri atau shared contract repository yang versioned.

```
service MovieService { rpc GetMovie(GetMovieRequest) returns (MovieResponse); rpc CreateMovie(CreateMovieRequest)
returns (MovieResponse); rpc UpdateMovie(UpdateMovieRequest) returns (MovieResponse); rpc
DeleteMovie(DeleteMovieRequest) returns (DeleteMovieResponse); rpc PublishMovie(PublishMovieRequest) returns
(MovieResponse); } service MediaService { rpc CreateUpload(CreateUploadRequest) returns (UploadResponse); rpc
```

```
GetJobStatus(JobStatusRequest) returns (JobStatusResponse); rpc StartTranscode(StartTranscodeRequest) returns (JobResponse); } service AuthService { rpc Login(LoginRequest) returns (LoginResponse); rpc Refresh(RefreshRequest) returns (RefreshResponse); rpc Revoke(RevokeRequest) returns (RevokeResponse); }
```

Gunakan timeout, retry terbatas, correlation/request ID, structured error code, dan health check pada setiap gRPC client/server.

13. Pemanfaatan Redis

- Cache katalog/category: TTL pendek-menengah dan invalidasi setelah mutation.
- Refresh-token/session revocation: simpan token identifier atau hash dengan TTL.
- Rate limiting login, CAPTCHA verification, dan API sensitif.
- Ad frequency/session state.
- Job queue/stream untuk transcoding dan event jika tidak menggunakan message broker khusus.
- Distributed lock untuk mencegah job transcoding yang sama dijalankan dua kali.

Redis bukan source of truth untuk data movie, user, atau transaksi penting; PostgreSQL tetap menjadi sumber data utama.

14. Security dan Hardening

- Semua koneksi production menggunakan HTTPS/TLS.
- JWT access token berumur pendek; refresh token menggunakan rotation/revocation.
- Password menggunakan Argon2id atau bcrypt dengan cost yang sesuai.
- CAPTCHA diterapkan terutama pada login berisiko, registrasi, reset password, atau setelah threshold percobaan gagal.
- Rate limit berdasarkan IP dan account identifier.
- Validasi input, content-type, ukuran file, MIME sniffing, dan path traversal.
- Authorization dicek di server; frontend hanya menyembunyikan UI, bukan menjadi mekanisme security.
- Audit log untuk create/update/delete movie, account, role, QRIS, ads, dan konfigurasi.
- Soft delete untuk entity penting bila histori perlu dipertahankan.
- Secrets disimpan melalui environment/secret manager, bukan hard-coded atau committed ke Git.

15. Docker dan Deployment

Development dapat menggunakan Docker Compose; production dapat dipindahkan ke Kubernetes/Nomad atau VM cluster sesuai skala.

Contoh service pada docker-compose:

```
services: frontend: api-gateway: auth-service: user-service: movie-service: category-service: media-service: subtitle-service: ad-service: support-service: audit-service: media-worker: postgres: redis: minio: # optional object storage nginx: # optional reverse proxy
```

Prinsip deployment:

- Gunakan multi-stage Docker build untuk binary Go kecil.
- Container berjalan sebagai non-root jika memungkinkan.
- Healthcheck dan graceful shutdown pada setiap service.
- Persistent volume untuk PostgreSQL, Redis sesuai kebutuhan, dan object storage.
- Backup PostgreSQL terjadwal dan uji restore secara berkala.
- Log dikirim ke centralized logging; metrics dan tracing dapat ditambahkan kemudian.

16. Struktur Frontend React JS

```
src/ app/ components/ layouts/ pages/ auth/ dashboard/ movies/ categories/ users/ ads/ support/ services/ api/ auth/ hooks/ store/ routes/ utils/ types/ assets/
```

Frontend menggunakan route guard berbasis role/permission, HTTP client terpusat, error handler, upload progress, HLS player, subtitle selector, quality selector, dan state management sesuai kebutuhan aplikasi.

17. Playback dan Streaming

Flow playback yang direkomendasikan:

- 1 Viewer meminta playback session.
- 2 Backend memvalidasi user/role dan status movie PUBLISHED.
- 3 Ad Service mengembalikan pre-roll jika campaign tersedia.
- 4 Frontend menjalankan ad lalu meminta/menjalankan media playlist.
- 5 Media Service mengeluarkan signed URL/short-lived playback token untuk HLS.
- 6 Player memilih quality berdasarkan bandwidth/device; user dapat override quality bila tersedia.
- 7 Playback events dicatat secara asynchronous agar tidak memperlambat streaming.

Signed URL atau playback token membantu mencegah URL media statis tersebar. Untuk konten berlisensi, DRM dan aturan hak distribusi perlu dipertimbangkan terpisah; jangan menganggap JWT saja sebagai DRM.

18. Struktur Repository Backend Go

```
backend/ cmd/ gateway/ auth/ user/ movie/ category/ media/ subtitle/ ad/ support/ audit/ worker/ internal/ domain/ application/ repository/ grpc/ http/ middleware/ config/ logger/ proto/ migrations/ deployments/ docker/ compose/ pkg/
```

Pisahkan domain/application/repository agar business rule tidak tercampur dengan handler HTTP atau gRPC. Repository menangani PostgreSQL; service/use-case menangani aturan bisnis; handler hanya mengorquestrasi request/response.

19. Observability dan Operasional

Area	Implementasi
Logging	Structured JSON log + request_id/correlation_id
Metrics	Request latency, error rate, active jobs, transcoding duration, Redis hit ratio
Tracing	OpenTelemetry untuk HTTP → gRPC → DB/Redis bila dibutuhkan
Alerting	5xx tinggi, DB connection issue, Redis unavailable, job FAILED meningkat
Audit	Aksi admin dan event keamanan
Backup	PostgreSQL backup + restore test

20. Tahapan Implementasi

Tahap	Output
1. Foundation	Repository, Docker, config, PostgreSQL migration, Redis, CI basic
2. Auth & RBAC	Login, CAPTCHA, JWT, refresh/revoke, middleware role
3. User	Profile, admin account management, status account
4. Category	CRUD category + cache
5. Movie	CRUD metadata, poster, catalog, publish flow
6. Media	Upload, storage, worker, FFmpeg, multi-quality HLS
7. Subtitle	SRT/VTX, language, default subtitle
8. Playback	Playback session, signed URL/token, quality selector

Tahap	Output
9. Advertisement	Campaign, creative, decision, impression/completion
10. QRIS	Upload QRIS, display support page
11. Dashboard	Role-based dashboard and metrics
12. Hardening	Security test, load test, backup/restore, monitoring

21. Acceptance Criteria

- Viewer dapat login dan memutar movie yang berstatus PUBLISHED.
- Login gagal ketika CAPTCHA invalid atau kredensial salah.
- Role tidak dapat menjalankan endpoint di luar permission.
- Admin/SuperAdmin dapat upload movie dan melihat progress/job transcoding.
- Output media tersedia sesuai profile yang berhasil diproses.
- Subtitle dapat dipilih saat playback.
- Iklan diputuskan server dan event impression/completion tercatat.
- SuperAdmin dapat mengelola akun dan role.
- SuperAdmin dapat upload/ubah QRIS aktif.
- Penghapusan entity penting tercatat pada audit log.
- Restart container tidak menyebabkan database kehilangan data.
- Backup PostgreSQL dapat direstore dalam environment pengujian.

22. Kesimpulan Arsitektur

Desain ini memisahkan domain berdasarkan service sehingga Movie, Media/Transcoding, Authentication, Subtitle, Advertisement, QRIS/Support, dan Audit dapat dikembangkan serta di-scale secara independen. React berkomunikasi ke Fiber Gateway, sementara service internal berkomunikasi melalui gRPC. PostgreSQL menjadi source of truth, Redis menjadi cache/state/queue ringan, dan object storage menjadi lokasi file media.

Prioritas teknis terbesar adalah pipeline media dan playback: upload asynchronous, FFmpeg worker, multi-resolution HLS, signed playback access, serta monitoring job. Untuk skala produksi, object storage dan CDN sebaiknya digunakan agar trafik video tidak membebani service Go.

Catatan legal/operasional: aplikasi streaming harus memastikan hak distribusi/licensing untuk film, musik, subtitle, dan materi iklan yang digunakan. Fitur teknis tidak menggantikan kewajiban lisensi atau kebijakan penyedia konten.